API Interaction Plan with GitLab

1. Agent Trigger (Inbound to FastAPI)

This is how GitLab wakes up the AI Agent when a developer interacts with it.

- **Action:** Developer assigns an issue to the AI Agent OR replies to a clarification comment.
- **GitLab Mechanism:** System Webhook (Issue events and Note events).
- **Agent Endpoint:** POST /gitlab-webhook (FastAPI Server)
- **Payload Processed:** Extracts project_id, issue_iid, description, and assignee_id.

2. Context Gathering Phase (Outbound to GitLab)

Once awake, the agent uses these endpoints to pull the necessary context to understand the assignment.

- **Fetch Full Issue Details:**
    - **Endpoint:** GET /api/v4/projects/:id/issues/:issue_iid
    - **Purpose:** To retrieve the full bug report, current labels, and existing comments.
- **Fetch Repository Tree (File List):**
    - **Endpoint:** GET /api/v4/projects/:id/repository/tree
    - **Purpose:** To map out the repository structure and locate relevant .md guidelines or source code files.
- **Fetch Specific File Content:**
    - **Endpoint:** GET /api/v4/projects/:id/repository/files/:file_path/raw
    - **Purpose:** To read the specific markdown (.md) files or Python/code files for context checking.

3. Clarification Loop (Outbound to GitLab)

If the agent's LiteLLM logic determines the context is insufficient (Step 4.1 in workflow).

- **Ask Clarifying Question:**
    - **Endpoint:** POST /api/v4/projects/:id/issues/:issue_iid/notes
    - **Purpose:** The agent posts a comment on the GitLab issue asking the developer for missing information (e.g., "Could you specify which file contains the login module?").

4. Execution & Delivery Phase (Outbound to GitLab)

If the context is sufficient, the agent proceeds to implementation (Steps 5 & 6 in workflow).

- **Set Issue Label:**
    - **Endpoint:** PUT /api/v4/projects/:id/issues/:issue_iid

    - **Purpose:** To automatically categorize the task by adding bug, feature, or hotfix to the labels array.
- **Create New Branch:**
    - **Endpoint:** POST /api/v4/projects/:id/repository/branches
    - **Purpose:** To create a dedicated working branch using TDET's strict naming rules (e.g., feature/issue-42-login-fix).
- **Commit Generated Code:**
    - **Endpoint:** POST /api/v4/projects/:id/repository/commits
    - **Purpose:** To push the AI-generated implementation and test files to the newly created branch in a single commit.
- **Create Merge Request:**
    - **Endpoint:** POST /api/v4/projects/:id/merge_requests
    - **Purpose:** To open the final MR for human review, including the self-review notes in the MR description, and linking it back to the original issue.

TDET AI Agent: Initial Milestone Breakdown

Milestone 1: Discovery & Architecture Setup (Weeks 1–4)

Covers Phase 1 & Phase 2

- **Weeks 1–2:** Understand GitLab workflows, research existing tools (GitLab Duo, Copilot), and clarify project scope.
- **Weeks 3–4 (Current):** * Finalize the High-Level Architecture Diagram.
    - Define the GitLab API Interaction Plan.
    - Successfully test the connection to the internal TDET Proxy Server (Azure AI Foundry) using LiteLLM.

Milestone 2: "Ears and Hands" - Core Integration (Weeks 5–8)

Covers Phase 3: Core Integration Prototype

- **Week 5:** Set up the FastAPI server to successfully catch and parse GitLab webhooks (Issue assignments/comments).
- **Week 6:** Implement GitLab API calls to automatically fetch the full issue descriptions and comments.
- **Week 7:** Implement GitLab API calls to pull repository context (fetching the project tree and reading specific .md and .py files).
- **Week 8:** Investigate external context collection (e.g., initial research into MCP servers or local vector search for repository guidelines).

Milestone 3: "The Brain" - AI Logic & Clarification Loop (Weeks 9–12)

Covers Phase 4: AI Agent Logic Implementation

- **Week 9:** Build the "Context Enough?" decision logic using LiteLLM and the gpt-main model.
- **Week 10:** Implement the automated clarification loop (Agent posts a question back to the GitLab issue if context is missing).
- **Week 11:** Implement automatic issue labeling (Agent categorizes the issue as bug, feature, or hotfix).
- **Week 12:** Finalize a highly stable system prompt structure for the agent to reliably process various edge cases without crashing.

Milestone 4: Execution & Sandboxing (Weeks 13–15)

Covers Phase 5: Code Generation & Validation

- **Week 13:** Automate GitLab branch creation with strict TDET naming conventions (e.g., feature/issue-name).

- **Week 14:** Develop the code generation prompts to output reliable Python code based on the issue context.
- **Week 15:** Implement basic test execution/validation to ensure the generated code runs correctly before committing.

Milestone 5: Automation & Handoff (Weeks 16 – End of September)

Covers Phase 6: Merge Request Automation & Hardening

- **Week 16:** Build the AI self-review logic (Agent checks its own generated code against the original issue requirements).
- **Week 17:** Automate the creation of the final Merge Request, injecting the AI's self-review notes into the MR description.
- **Final Weeks:** System hardening, handling corporate policy/firewall edge cases, and finalizing project documentation for the TDET team.

Proposal 1: Use LangGraph to Solve the "Agent Mode" Policy Block

Your mentors mentioned they had issues running GitHub Copilot in "agent mode" due to corporate policies. This usually happens because corporate firewalls detect a script making rapid, endless loops of API calls and flag it as a security risk (like a DDoS attack).

- **The Suggestion:** Propose using **LangGraph** (a library specifically for building stateful, multi-agent applications) instead of writing custom while loops in Python.
- **Why it's an improvement:** LangGraph allows you to define a maximum number of steps (e.g., the agent can only ask clarifying questions a maximum of 3 times before stopping). It also natively supports **Human-in-the-Loop (HITL)**. If the corporate policy blocks autonomous execution, LangGraph can pause the script, send a Teams message or terminal prompt to a human saying *"I plan to write this code, approve? (Y/N)"*, and only continue once approved. It perfectly bypasses "runaway agent" security blocks.

Proposal 2: Use ChromaDB + Azure Embeddings for the Repository Context

In the diagram's hints, your mentors wrote: *"AI should action follow .md in each repo."* If a repository has 50 different Markdown guidelines, you can't stuff all of them into a single LiteLLM prompt—it will crash or cost too much money.

- **The Suggestion:** Propose implementing a lightweight **RAG (Retrieval-Augmented Generation)** pipeline using **ChromaDB** (a free, local vector database).
- **Why it's an improvement:** Instead of the agent guessing which .md file to read, the agent will convert the user's issue into a search query, and ChromaDB will instantly fetch only the 2 or 3 most relevant paragraphs from the repository's documentation. Since TDET already uses Azure AI Foundry, you can use Azure's embedding models to do this completely securely within the corporate network!

Proposal 3: Use Docker Sandboxing for Safe Code Testing

Step 5 of your workflow says the AI Agent will *"implement and test"* the feature. Running AI-generated Python or shell scripts directly on the TDET server is a massive security risk. If the AI hallucinates and writes a command like rm -rf / (delete everything), it could wipe out files.

- **The Suggestion:** Propose adding **Docker SDK for Python** into your tech stack.
- **Why it's an improvement:** Before the agent commits the code, your Python script can spin up an isolated, temporary Docker container. The agent runs its tests inside that container. If the code crashes or does something destructive, it only destroys the disposable container, keeping the main TDET server and your laptop 100% safe.

Proposal: AI Agent Project Task Division Strategies

Having two interns on the TDET AI Agent project provides a significant advantage. To ensure smooth collaboration and maximize learning, we have outlined five potential strategies for dividing the project tasks. We would like to present these options to the mentorship team to decide the best path forward.

Option 1: The "Component" Split (Infrastructure vs. AI Logic)

This strategy divides the workload by technical domain, preventing merge conflicts by giving each intern ownership over a specific piece of the tech stack.

- **Intern A (Ears and Hands):** FastAPI webhook listener, GitLab API integration, branch/MR management.
- **Intern B (The Brain):** LiteLLM integration, decision logic, prompt engineering, self-review logic.

Option 2: The "Pipeline" Split (Front-Half vs. Back-Half)

This strategy divides the workload by the project's chronological workflow. This provides "full-stack" experience, as both interns work with both the GitLab API and LiteLLM.

- **Intern A (Intake & Clarification):** Webhook handling, context collection, context check, developer clarification loop.
- **Intern B (Execution & Delivery):** Issue labeling, code generation, test execution, self-review, MR creation.
- *Note: Requires a "Mock Data" contract to work in parallel.*

Option 3: The "MVP vs. Extensions" Split (Core vs. Advanced)

One intern builds the foundational End-to-End agent, while the other builds the advanced architecture requested by mentors.

- **Intern A (Core MVP Builder):** Builds the base pipeline (FastAPI/LiteLLM/GitLab).
- **Intern B (Advanced Systems Architect):** Builds RAG (ChromaDB), Docker Sandboxing, and LangGraph.

Option 4: Agile Pair Programming

Both interns work on the exact same codebase simultaneously using VS Code Live Share.

- **Process:** Interns alternate between "Driver" (typing code) and "Navigator" (reviewing architecture, logic, and debugging).
- **Benefit:** 100% full-stack experience for both, high code quality, and deep team collaboration.

Option 5: Capability-Based Split (Understanding vs. Acting)

This split focuses on the two most difficult problems in agent systems: **Understanding** the repository well enough to plan, and **Acting** accurately to implement and validate code. This is the recommended approach for tackling complex enterprise agentic systems.

Intern A: Repository Intelligence (Understanding)	Intern B: Execution Intelligence (Acting)
• Context retrieval (RAG / ChromaDB). • MCP Server integration. • Context sufficiency analysis. • Clarification loop.	• Code generation prompts. • Test generation (Docker validation). • Self-review logic. • MR generation.

Shared Tech Stack: FastAPI webhook handling, GitLab API requests, system architecture, and project documentation.

Summary

While all options provide a pathway to success, the **Capability-Based Split (Option 5)** is our top recommendation as it allows both interns to gain full-stack experience while directly addressing the hardest technical hurdles of our agent architecture.